Bases de Datos Distribuidas

Práctica Final: Bases de Datos Distribuidas para una Compañía Vinícola

Lucía Cepeda González y Adriana Rubia Alcarria

Curso 2025/26

Índice

1. Enunciado de la pŕactica **3**

2. Práctica 1: Diseño conceptual y lógico global de la base de datos **5**
2.1. Diagrama entidad-relación 5
2.2. Paso a tablas 7
2.2.1. Fusión de tablas 8
2.2.2. Esquema lógico global 9

3. Práctica 2: Diseño de la fragmentación de la asignación **10**
3.1. Fragmentación de tablas 10
3.2. Réplica de fragmentos y relaciones 14
3.3. Asignación de fragmentos, réplicas y relaciones 14
3.4. Reconstrucción de las relaciones globales a partir de los fragmentos 15

4. Práctica 3: Implementación del diseño **15**
4.1. Código de creación de tablas y vistas 15

5. Práctica 4: Implementación de actualizaciones **15**
5.1. Procedimientos en PL/SQL para operaciones de actualización 15

6. Práctica 5: Implementación de consultas **15**

1. Enunciado de la pŕactica

Una compañía vinícola española desea implementar una base de datos distribuida para gestionar, lo más eficientemente posible, la distribución de vinos españoles por todo el territorio nacional. La compañía dispone de cuatro delegaciones territoriales: **Madrid** (localidad 1), **Barcelona** (localidad 2), **La Coruña** (localidad 3) y **Granada** (localidad 4). La delegación de Madrid suministra vinos a clientes de la Región Centro de la península (Castilla-León, Castilla-La Mancha, Aragón, Madrid, La Rioja). La delegación de Barcelona suministra vinos a clientes del Levante Español (Cataluña, Baleares, País Valenciano y Murcia). La delegación de La Coruña suministra vinos a clientes del Norte de España (Galicia, Asturias, Cantabria, País Vasco y Navarra). Y, por último, la delegación de Granada suministra vinos a clientes de la Zona Sur del país (Andalucía, Extremadura, Canarias, Ceuta y Melilla). Cada delegación realiza sus labores de gestión a través de sucursales ubicadas en diferentes ciudades de cada autonomía. Cada sucursal solamente puede distribuir directamente vinos producidos en las comunidades autónomas que pertenecen a su delegación. Un cliente puede solicitar suministros de cualquier vino, pero siempre tendrá que hacerlo a través de sucursales de la delegación a la que pertenece su comunidad autónoma. Si una sucursal no puede suministrar directamente un vino a un cliente porque su delegación no lo distribuye, esta sucursal solicitará el pedido de vino a cualquier sucursal de la delegación que sí lo distribuya. Por ejemplo, un cliente de Andalucía requiere suministros de vinos de La Rioja. Este cliente tendrá que solicitar su pedido a cualquier sucursal de la delegación de Granada. Puesto que esta delegación no distribuye vinos de la Rioja, la sucursal a la que se haya dirigido el cliente tendrá que solicitar el pedido a una sucursal de la delegación de Madrid que es la encargada de la distribución de los vinos de la Rioja.

Los principales requisitos de funcionamiento y de almacenamiento de la compañía se describen con la siguiente lista de especificaciones:

LOS CLIENTES de la compañía se identifican mediante un código de cliente y, para cada uno de ellos, se almacena el DNI (o CIF), nombre del cliente, dirección o domicilio social del cliente, tipo de cliente, comunidad autónoma en la que reside, los distintos suministros de vino que la distribuidora le va a proporcionar (o le ha proporcionado) y las sucursales que van a tramitar(o han tramitado) cada uno de esos suministros. Cada suministro consiste en un tipo de vino, cantidad que se va a suministrar (número de botellas) y la fecha en la que se solicitó el suministro. A efectos de almacenamiento, la solicitud de dos (o más) suministros del mismo vino en la misma fecha y requeridos a la misma sucursal por el mismo cliente, se consideran como un único suministro en el cual se suman las cantidades que se van a suministrar (o se han suministrado).

LOS EMPLEADOS de la compañía se identifican mediante un código de empleado, código que mantendrán mientras trabajen en dicha compañía independientemente de la sucursal en la que estén destinados en un momento determinado. La administración de ladistribuidora

almacena para cada empleado el DNI, nombre del empleado, fecha de comienzo de contrato con la compañía (NO la fecha de comienzo de trabajo en una sucursal), salario, dirección y sucursal de destino. Cada empleado, en un momento dado, sólo puede estar asignado en una sucursal (no se mantendrá registro de las distintas sucursales a las que haya estado destinado el empleado).

LOS VINOS se identifican mediante un código de vino y, para cada uno de ellos, se almacena la marca del vino, año de la cosecha, denominación de origen (si la tiene), graduación, viñedo de procedencia, comunidad autónoma, cantidad producida (número de botellas), cantidad en stock (número de botellas) y productor. En cada momento, la cantidad en stock será la cantidad producida menos la cantidad total suministrada. Inicialmente, la cantidad en stock es la cantidad producida.

LOS PRODUCTORES se identifican mediante un código de productor y, para cada uno de ellos, se almacena el DNI (o CIF), nombre del productor, dirección o domicilio social del productor y los vinos que produce.

LAS SUCURSALES se identifican mediante un código de sucursal y, para cada una de ellas, se almacena el nombre de la sucursal, ciudad en la que se encuentra, comunidad autónoma en la que se ubica, director de la sucursal y los pedidos que ha realizado a otras sucursales para tramitar los suministros solicitados por sus clientes. Cada pedido consiste, en un tipo de vino, cantidad que se solicita y fecha del pedido. A efectos de almacenamiento, dos (o más) pedidos del mismo vino en la misma fecha y solicitados por la misma sucursal, se consideran como un único pedido en el cual se suman las cantidades solicitadas.

Las aplicaciones, tanto de actualización como de consulta, que usan datos de sucursales (incluidos datos de empleados, clientes, suministros, vinos que distribuye, etc.), se pueden generar en cualquier localidad, pero referencian con una probabilidad del 0,95 a sucursales cercanas.

2. Práctica 1: Diseño conceptual y lógico global de la base de datos

2.1. Diagrama entidad-relación

El principal dilema que tuvimos al diseñar el diagrama entidad-relación del enunciado fue sobre el concepto de "suministro". Al principio pensamos en dos opciones: considerar suministro como una entidad, o considerar suministro como una relación de argregación. Nos dimos cuenta de que el enunciado especifica que "la solicitud de dos (o más) *suministros* del mismo *vino* en la misma *fecha* y requeridos a la misma *sucursal* por el mismo *cliente*, se consideran como un único suministro", conque un suministro requiere información de tres de las entidades: sucursal, cliente y vino. Además de requerir una fecha, la cual se puede añadir como atributo clave parcial de una relación. Como sabemos, al pensar en una relación que podría ser terciaria, la solución es usar una agregación. Decidimos elegir la opción de representar suministro como una agregación, ya que representa perfectamente el concepto y consigue relacionar las tres entidades cumpliendo el requerimiento del enunciado. Haremos una agregación "sucursal - vende - vino" y esta se relaciona a través de la relación "suministro" con la entidad "cliente". Teniendo, por supuesto, una clave parcial "fecha" en "suministro".

Tras esta decisión, nos percatamos de que la relación "pedido", que se refiere a pedidos que pueden hacer unas sucursales a otras, podría no ser simplemente una relación recursiva, como habíamos pensado en un principio. Ya que, al igual que la relación subimistro, necesita también información sobre el vino que se está pidiendo. Al ser una situación tan similar a la anterior, creemos que la mejor solución es de nuevo hacer de "pedido" una relación de agregación. De hecho, es la misma agregación que suministro. Con la agregación que ya hemos creado de "sucursal - vende - vino", se relaciona "pedido" con "sucursal" de nuevo. Teniendo igualmente una clave parcial "fecha" en "pedido". De este modo, la agregación estará relacionada con dos entidades distintas. Esto es aceptable, ya que la agregación es una tabla en sí, y puede estar relacionada con todas las entidades que se precise.

A continuación se expone el diagrama Entidad-Relación resultante del análisis del problema.

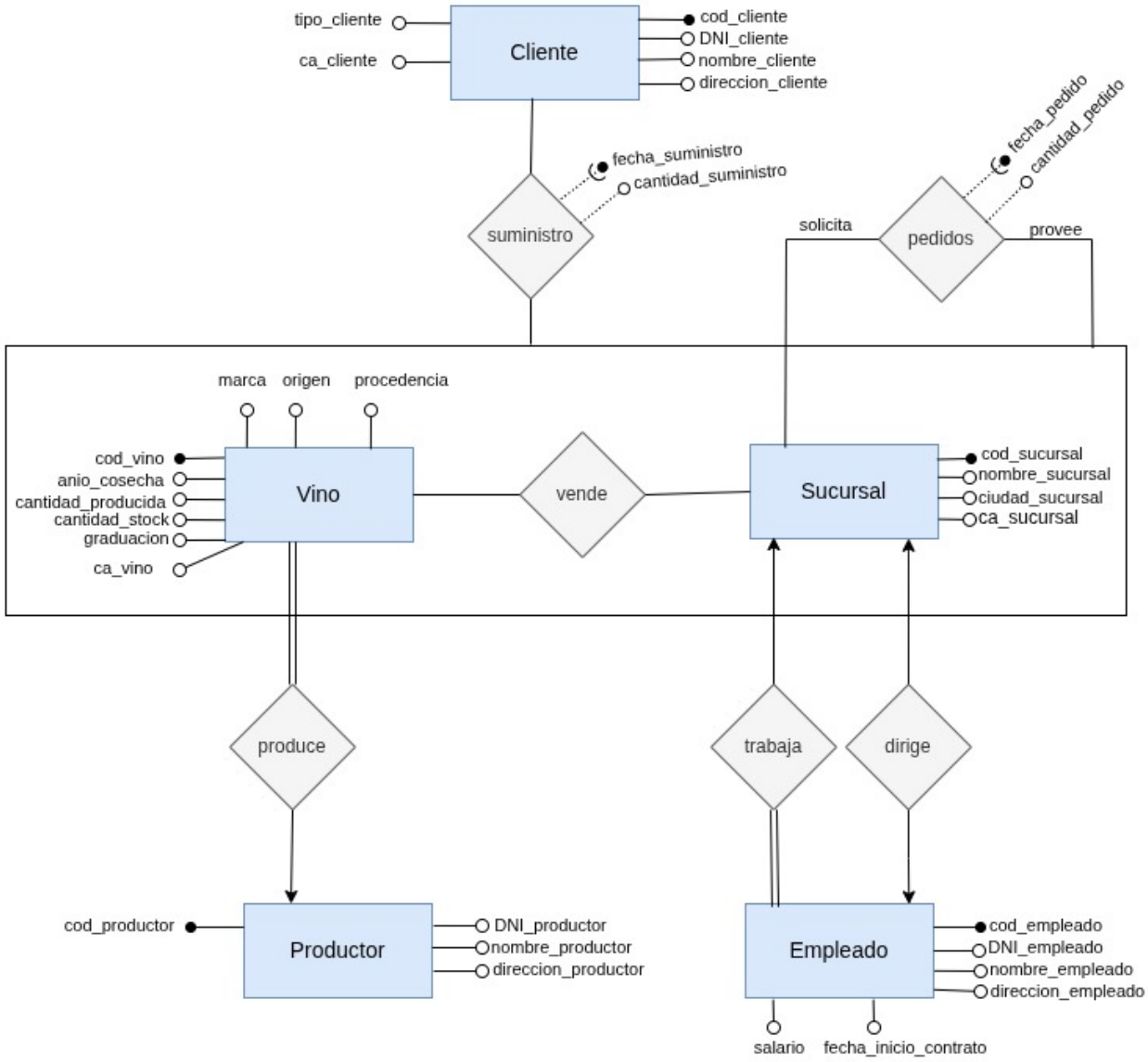

Figura 1: Diagrama E/R

2.2. Paso a tablas

Realizamos el paso a tablas del modelo de datos relacional descrito en el apartado anterior.

Entidades:

- *Cliente* (cod_cliente, DNI_cliente, nombre_cliente, direccion_cliente, tipo_cliente, ca_cliente)
- *Sucursal* (cod_sucursal, nombre_sucursal, ciudad_sucursal, ca_sucursal)
- *Vino* (cod_vino, marca, anio_cosecha, origen, graduacion, procedencia, cantidad_producida, cantidad_stock, ca_vino)
- *Empleado* (cod_empleado, DNI_empleado, nombre_empleado, direccion_empleado, fecha_inicio_contrato, salario)
- *Productor* (cod_productor, DNI_productor, nombre_productor, direccion_productor)

Relaciones:

- *VinoProducido* (cod_vino, cod_productor)
  $cod_vino \rightarrow CE(Vino)$ *y* $cod_productor \rightarrow CE(Productor)$
- *EmpleadoTrabajaSucursal* (cod_empleado, cod_sucursal)
  $cod_empleado \rightarrow CE(Empleado)$ *y* $cod_sucursal \rightarrow CE(Sucursal)$
- *EmpleadoDirigeSucursal* (cod_sucursal, cod_empleado)
  $cod_sucursal \rightarrow CE(Sucursal)$ *y* $cod_empleado \rightarrow CE(Empleado)$
- *SucursalVendeVino* (cod_sucursal, cod_vino)
  $cod_vino \rightarrow CE(Vino)$ *y* $cod_sucursal \rightarrow CE(Sucursal)$
- *ClienteSolicitaSuministro* (cod_vino, cod_sucursal, cod_cliente, fecha_sumimistro, cantidad_suministro)
  $cod_vino, cod_sucursal \rightarrow CE(SucursalVendeVino)$ *y* $cod_cliente \rightarrow CE(Cliente)$
- *SucursalPideSuministro* (cod_sucursal_solicita, cod_vino, cod_sucursal_provee, fecha_pedido, cantidad_pedido)

$(cod_sucursal_solicita, cod_vino) \rightarrow CE(SucursalVendeVino)$ y
$cod_sucursal_provee \rightarrow CE(Sucursal)$

2.2.1. Fusión de tablas

- *Sucursal* (cod_sucursal, nombre_sucursal, ciudad_sucursal, ca_sucursal, cod_director*)
- *EmpleadoDirigeSucursal* (cod_sucursal, cod_empleado)
  $cod_sucursal \rightarrow CE(Sucursal)$ y $cod_empleado \rightarrow CE(Empleado)$

Usamos propagación en la inclusión del campo cod_empleado en la relación Sucursal (una sucursal es dirigida por un único empleado). De esta forma no es necesario modelar la relación EmpleadoDirigeSucursal.

- *Empleado* (cod_empleado, DNI_empleado, nombre_empleado, direccion_empleado, fecha_inicio_contrato, salario, cod_sucursal*)
- *EmpleadoTrabajaSucursal* (cod_empleado, cod_sucursal)
  $cod_empleado \rightarrow CE(Empleado)$ y $cod_sucursal \rightarrow CE(Sucursal)$

Usamos propagación en la inclusión del campo cod_sucursal en la relación Empleado (un empleado trabaja en una única sucursal). De esta forma no es necesario modelar la relación EmpleadoTrabajaSucursal.

- *Vino* (cod_vino, marca, anio_cosecha, origen, graduacion, procedencia, cantidad_producida, cantidad_stock, ca_vino, cod_productor*)
- *VinoProducido* (cod_vino, cod_productor)
  $cod_vino \rightarrow CE(Vino)$ y $cod_productor \rightarrow CE(Productor)$

Usamos propagación en la inclusión del campo cod_productor en la relación Vino (un vino es producido por un único productor). De esta forma no es necesario modelar la relación VinoProducido.

2.2.2. Esquema lógico global

Así queda el esquema lógico global. La Figura 2 representa las tablas y cómo se relacionan entre ellas, con sus claves primarias y claves externas.

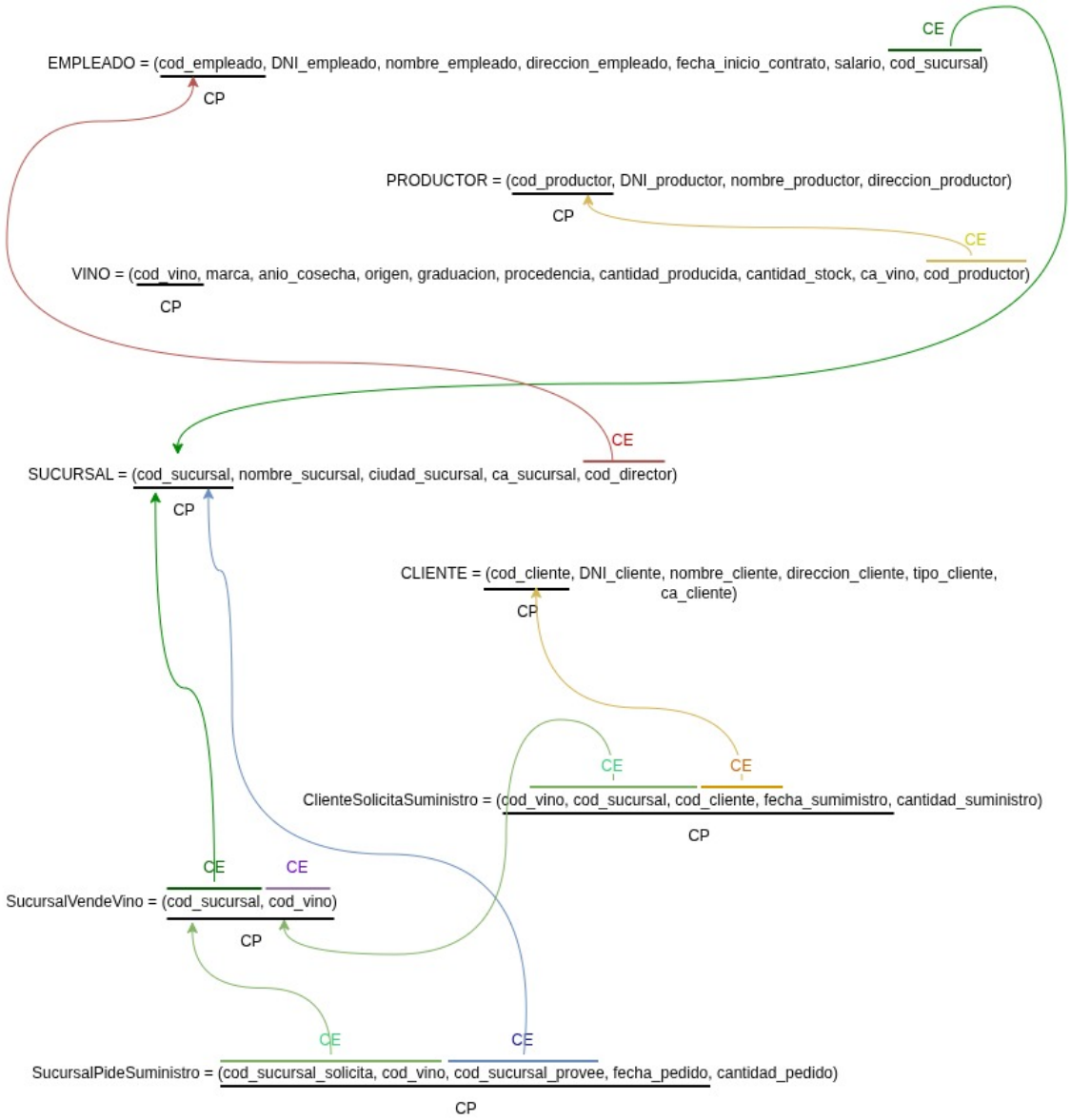

Figura 2: Esquema lógico global

3. Práctica 2: Diseño de la fragmentación de la asignación

3.1. Fragmentación de tablas

En primer lugar, fragmentaremos la relación *Sucursal* de forma **horizontal primaria** a partir del atributo **Comunidad Autónoma** (ca_sucursal).

Una alternativa podría consistir en añadir un nuevo atributo denominado "localidad" y fragmentar la relación en función de dicho valor. Sin embargo, consideramos que siguiendo la secuencia lógica de diseño de una base de datos distribuida, esto supondría anticipar decisiones propias del diseño físico, ya que *localidad* alude directamente a la ubicación física de los datos.

En esta fase del diseño distribuido, lo correcto es fragmentar basándonos en atributos lógicos del dominio, como ca_suc, que reflejan la organización conceptual del negocio y no la localización de los fragmentos. A partir de este atributo obtenemos un **conjunto completo y minimal de predicados simples** que permitirá definir los fragmentos horizontales de manera formal.

Conjunto P de predicados simples:

```
P = {ca_sucursal="CastillaLeon", ca_sucursal="Madrid",
     ca_sucursal="CastillaLaMancha", ca_sucursal="Aragon",
     ca_sucursal="LaRioja", ca_sucursal="Cataluña",
     ca_sucursal="Baleares", ca_sucursal="País Valenciano",
     ca_sucursal="Murcia", ca_sucursal="Galicia",
     ca_sucursal="Asturias", ca_sucursal="Cantabria",
     ca_sucursal="País Vasco", ca_sucursal="Navarra",
     ca_sucursal="Andalucía", ca_sucursal="Extremadura",
     ca_sucursal="Canarias", ca_sucursal="Ceuta",
     ca_sucursal="Melilla"}
```

Por simplicidad, usaremos la siguiente notación:

$$p_1 \equiv \texttt{ca_sucursal="CastillaLeon"}$$
$$p_2 \equiv \texttt{ca_sucursal="Madrid"}$$
$$p_3 \equiv \texttt{ca_sucursal="CastillaLaMancha"}$$
$$p_4 \equiv \texttt{ca_sucursal="Aragon"}$$
$$p_5 \equiv \texttt{ca_sucursal="LaRioja"}$$
$$p_6 \equiv \texttt{ca_sucursal="Cataluña"}$$

$$
\begin{aligned}
p_7 &\equiv \texttt{ca_sucursal="Baleares"} \\
p_8 &\equiv \texttt{ca_sucursal="País Valenciano"} \\
p_9 &\equiv \texttt{ca_sucursal="Murcia"} \\
p_{10} &\equiv \texttt{ca_sucursal="Galicia"} \\
p_{11} &\equiv \texttt{ca_sucursal="Asturias"} \\
p_{12} &\equiv \texttt{ca_sucursal="Cantabria"} \\
p_{13} &\equiv \texttt{ca_sucursal="País Vasco"} \\
p_{14} &\equiv \texttt{ca_sucursal="Navarra"} \\
p_{15} &\equiv \texttt{ca_sucursal="Andalucía"} \\
p_{16} &\equiv \texttt{ca_sucursal="Extremadura"} \\
p_{17} &\equiv \texttt{ca_sucursal="Canarias"} \\
p_{18} &\equiv \texttt{ca_sucursal="Ceuta"} \\
p_{19} &\equiv \texttt{ca_sucursal="Melilla"}
\end{aligned}
$$

Tenemos por tanto $2^{19} = 524288$ expresiones conjuntivas.

$$
\begin{aligned}
&y_1 = p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge \cdots \wedge \neg p_{19} \Rightarrow \text{Verdadero} \\
&y_2 = \neg p_1 \wedge p_2 \wedge \neg p_3 \wedge \cdots \wedge \neg p_{19} \Rightarrow \text{Verdadero} \\
&y_3 = \neg p_1 \wedge \neg p_2 \wedge p_3 \wedge \cdots \wedge \neg p_{19} \Rightarrow \text{Verdadero} \\
&y_4 = \neg p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge p_4 \wedge \cdots \wedge \neg p_{19} \Rightarrow \text{Verdadero} \\
&\vdots \\
&y_{19} = \neg p_1 \wedge \neg p_2 \wedge \neg p_3 \wedge \neg p_4 \wedge p_5 \wedge \cdots \wedge p_{19} \Rightarrow \text{Verdadero}
\end{aligned}
$$

Debido al alto número de expresiones conjuntivas, hemos numerado únicamente las verdaderas, las expresiones en las que aparecen todas negadas salvo una. El resto de expresiones son falsas pues la comunidad autónoma a la que pertenece una sucursal es única. Por ello, cualquier expresión que involucre la pertenencia simultánea de una sucursal a dos o más comunidades autónomas distintas es falsa. En consecuencia, **los términos de predicado** serán $y_1, y_2, y_3, \ldots, y_{19}$.

Teniendo en cuenta el proceso anterior, el esquema de fragmentación para la relación *Sucursal* es el siguiente:

$$
\begin{aligned}
&\text{Sucursal}_1 = \text{SL}_{y_1}(\text{Sucursal})\\
&\text{Sucursal}_2 = \text{SL}_{y_2}(\text{Sucursal})\\
&\text{Sucursal}_3 = \text{SL}_{y_3}(\text{Sucursal})\\
&\text{Sucursal}_4 = \text{SL}_{y_4}(\text{Sucursal})\\
&\vdots\\
&\text{Sucursal}_{19} = \text{SL}_{y_{19}}(\text{Sucursal})
\end{aligned}
$$

Puesto que cada sucursal solamente puede distribuir directamente vinos producidos en las comunidades autónomas que pertenecen a su delegación, y los clientes pueden solicitar suministros de cualquier vino, pero siempre a través de sucursales de la delegación a la que pertenece su comunidad autónoma, hemos procedido de forma análoga al proceso anterior.

Los esquemas de fragmentación resultantes para las relaciones *Vino* y *Cliente* son los siguientes:

$$
\begin{aligned}
&\text{Vino}_1 = \text{SL}_{y_1}(\text{Vino})\\
&\text{Vino}_2 = \text{SL}_{y_2}(\text{Vino})\\
&\text{Vino}_3 = \text{SL}_{y_3}(\text{Vino})\\
&\text{Vino}_4 = \text{SL}_{y_4}(\text{Vino})\\
&\vdots\\
&\text{Vino}_{19} = \text{SL}_{y_{19}}(\text{Vino})
\end{aligned}
$$

$$
\begin{aligned}
&\text{Cliente}_1 = \text{SL}_{y_1}(\text{Cliente})\\
&\text{Cliente}_2 = \text{SL}_{y_2}(\text{Cliente})\\
&\text{Cliente}_3 = \text{SL}_{y_3}(\text{Cliente})\\
&\text{Cliente}_4 = \text{SL}_{y_4}(\text{Cliente})\\
&\vdots\\
&\text{Cliente}_{19} = \text{SL}_{y_{19}}(\text{Cliente})
\end{aligned}
$$

Puesto que las aplicaciones requieren información sobre los empleados que trabajan en una sucursal y los vinos que distribuye una sucursal, habrá que facilitar las operaciones de producto natural entre estas relaciones.

De esta forma, las relaciones Empleado y SucursalVendeVino serán fragmentadas de manera horizontal derivada a partir de la fragmentación horizontal primaria de Sucursal.

$$
\begin{aligned}
\text{Empleado}_1 &= \text{Empleado SJN}_{E.cod_sucursal=S.cod_sucursal}\ (\text{Sucursal}_1)\\
\text{Empleado}_2 &= \text{Empleado SJN}_{E.cod_sucursal=S.cod_sucursal}\ (\text{Sucursal}_2)\\
\text{Empleado}_3 &= \text{Empleado SJN}_{E.cod_sucursal=S.cod_sucursal}\ (\text{Sucursal}_3)\\
\text{Empleado}_4 &= \text{Empleado SJN}_{E.cod_sucursal=S.cod_sucursal}\ (\text{Sucursal}_4)\\
&\vdots\\
\text{Empleado}_{19} &= \text{Empleado SJN}_{E.cod_sucursal=S.cod_sucursal}\ (\text{Sucursal}_{19})
\end{aligned}
$$

$$
\begin{aligned}
\text{SucursalVendeVino}_1 &= \text{SucursalVendeVino SJN}_{S.cod_sucursal=S.cod_sucursal}\ (\text{Sucursal}_1)\\
\text{SucursalVendeVino}_2 &= \text{SucursalVendeVino SJN}_{S.cod_sucursal=S.cod_sucursal}\ (\text{Sucursal}_2)\\
\text{SucursalVendeVino}_3 &= \text{SucursalVendeVino SJN}_{S.cod_sucursal=S.cod_sucursal}\ (\text{Sucursal}_3)\\
\text{SucursalVendeVino}_4 &= \text{SucursalVendeVino SJN}_{S.cod_sucursal=S.cod_sucursal}\ (\text{Sucursal}_4)\\
&\vdots\\
\text{SucursalVendeVino}_{19} &= \text{SucursalVendeVino SJN}_{S.cod_sucursal=S.cod_sucursal}\ (\text{Sucursal}_{19})
\end{aligned}
$$

Por último, la aplicación requiere información sobre los los suministros de vino que un cliente solicita a una sucursal, por lo tanto, la relación ClienteSolicitaSuministro será fragmentada de forma horizontal derivada a partir de la fragmentación horizontal derivada SucursalVendeVino.

$$
\begin{aligned}
\text{ClienteSolicitaSuministro}_1 &= \text{SJN}_{C.cod_sucursal=S.cod_sucursal}\ (\text{SucursalVendeVino}_1)\\
\text{ClienteSolicitaSuministro}_2 &= \text{SJN}_{C.cod_sucursal=S.cod_sucursal}\ (\text{SucursalVendeVino}_2)\\
\text{ClienteSolicitaSuministro}_3 &= \text{SJN}_{C.cod_sucursal=S.cod_sucursal}\ (\text{SucursalVendeVino}_3)\\
\text{ClienteSolicitaSuministro}_4 &= \text{SJN}_{C.cod_sucursal=S.cod_sucursal}\ (\text{SucursalVendeVino}_4)\\
&\vdots\\
\text{ClienteSolicitaSuministro}_{19} &= \text{SJN}_{C.cod_sucursal=S.cod_sucursal}\ (\text{SucursalVendeVino}_{19})
\end{aligned}
$$

3.2. Réplica de fragmentos y relaciones

Hemos decidido no replicar ningún fragmento, ya que cada uno pertenece a una delegación que atiende regiones específicas y las consultas se realizan mayoritariamente desde sucursales cercanas. La replicación aumentaría el coste y la complejidad sin aportar mejoras significativas en disponibilidad o rendimiento.

En cambio, replicamos la tabla **Productor** (una copia en cada localidad) para tener toda la información de los productores localmente puesto que los productores pueden producir vinos de diferentes comunidades autónomas. Además, teóricamente dicha tabla no debería actualizarse demasiado.

Asimismo, replicamos la tabla **SucursalPideSuministro** (una copia en cada localidad) para disponer de toda la información sobre los vinos que una sucursal solicita a otra, de modo que cada sucursal tenga localmente todo lo necesario para realizar el pedido.

3.3. Asignación de fragmentos, réplicas y relaciones

En esta sección se procede a la asignación de los distintos fragmentos a cada localidad, considerando que cada fragmento corresponde a una delegación que abarca varias regiones del territorio. Además, se tiene en cuenta que las aplicaciones acceden con una probabilidad del 0,95 a las sucursales más cercanas, lo que guía la ubicación preferente de cada fragmento.

- Sucursal_i, Vino_i, Cliente_i, Empleado_i, $\text{ClienteSolicitaSuministro}_i$, $\text{SucursalVendeVino}_i$ con $i \in \{1, 2, 3, 4, 5\} \Rightarrow$ Se asignan a la localidad de **Madrid**

- Sucursal_i, Vino_i, Cliente_i, Empleado_i, $\text{ClienteSolicitaSuministro}_i$, $\text{SucursalVendeVino}_i$ con $i \in \{6, 7, 8, 9\} \Rightarrow$ Se asignan a la localidad de **Barcelona**

- Sucursal_i, Vino_i, Cliente_i, Empleado_i, $\text{ClienteSolicitaSuministro}_i$, $\text{SucursalVendeVino}_i$ con $i \in \{10, 11, 12, 13, 14\} \Rightarrow$ Se asignan a la localidad de **La Coruña**

- Sucursal_i, Vino_i, Cliente_i, Empleado_i, $\text{ClienteSolicitaSuministro}_i$, $\text{SucursalVendeVino}_i$ con $i \in \{15, 16, 17, 18, 19\} \Rightarrow$ Se asignan a la localidad de **Granada**

3.4. Reconstrucción de las relaciones globales a partir de los fragmentos

Al ser todas las fragmentaciones horizontales o derivadas de fragmentaciones horizontales, las relaciones globales se pueden reconstruir a partir de los fragmentos mediante el operador de unión.

- $\text{Sucursal} = \text{Sucursal}_1 \bigcup \text{Sucursal}_2 \bigcup \ldots \bigcup \text{Sucursal}_{19}$
- $\text{Vino} = \text{Vino}_1 \bigcup \text{Vino}_2 \bigcup \ldots \bigcup \text{Vino}_{19}$
- $\text{Cliente} = \text{Cliente}_1 \bigcup \text{Cliente}_2 \bigcup \ldots \bigcup \text{Cliente}_{19}$
- $\text{Empleado} = \text{Empleado}_1 \bigcup \text{Empleado}_2 \bigcup \ldots \bigcup \text{Empleado}_{19}$
- $\text{ClienteSolicitaSuministro} = \bigcup_{i=1}^{19} \text{ClienteSolicitaSuministro}_i$
- $\text{SucursalVendeVino} = \bigcup_{i=1}^{19} \text{SucursalVendeVino}_i$

4. Práctica 3: Implementación del diseño

4.1. Código de creación de tablas y vistas

Implementación del diseño, haciendo uso de disparadores. Teniendo en cuenta unas restricciones de integridad.

El código correspondiente se encuentra impreso en las siguientes páginas.

5. Práctica 4: Implementación de actualizaciones

5.1. Procedimientos en PL/SQL para operaciones de actualización

Se han implementado una serie de procedimientos disponibles en el código de la práctica.

6. Práctica 5: Implementación de consultas

El código de las consultas se encuentra en las siguientes páginas.